



Universidad Politécnica de Querétaro

Ingeniería en Sistemas computacionales

Materia: Programación web

“Mercy Food – Sprint 4”

Docente: José Javier Moya Moya

Por: Andrés Joshua León Barranco

Grupo: S-204

31/oct/2025

1. Introducción

Mercy Food es una plataforma web diseñada para crear un ecosistema de delivery de comida que conecta a tres tipos de usuarios: clientes, restaurantes locales y repartidores independientes. El objetivo del proyecto es fortalecer la economía local a través de una aplicación moderna, segura y fácil de usar, que gestiona el proceso completo desde el registro de usuarios hasta la entrega de pedidos.

El proyecto ha evolucionado a través de varios sprints, comenzando con las bases de la autenticación, migrando a una arquitectura más robusta, implementando el ciclo de vida completo del pedido y, finalmente, refinando los perfiles de usuario y añadiendo módulos de soporte.

2. Arquitectura y tecnologías

Inicialmente, el backend del proyecto se desarrolló en Python (Flask), pero para asegurar la escalabilidad, se realizó una migración fundamental a **PHP 8.2** utilizando el framework **Laravel 12**.

Esta arquitectura sigue un patrón Modelo-Vista-Controlador (MVC) y aprovecha las siguientes herramientas clave:

- **Backend (Laravel):**
 - **ORM Eloquent:** para la gestión de la base de datos (modelos como User, Platillo, Pedido, Faq).
 - **Motor de plantillas Blade:** para la creación de vistas dinámicas del lado del servidor.
 - **Sistema de rutas (web.php):** para gestionar todos los endpoints de la aplicación.
 - **Localización (i18n):** uso de archivos de lenguaje (lang/es) para mensajes de validación y texto del sistema en español.
- **Frontend:**
 - **HTML5, CSS3 y JavaScript** vainilla.
 - **AJAX (Asynchronous JavaScript and XML):** utilizado intensivamente para la carga asíncrona de contenido en los dashboards. Esto ofrece una experiencia de usuario fluida, similar a una *Single Page Application* (SPA), donde la página no necesita recargarse completamente al navegar.
- **Base de datos:**
 - Se utiliza un gestor de base de datos relacional (como MySQL o SQL Server), gestionado a través del sistema de **migraciones** de Laravel.
 - Se utilizan **Seeders** (UserSeeder, PlatilloSeeder, FaqSeeder) para poblar la base de datos con datos de prueba.

3. Características y módulos principales

El desarrollo de la plataforma ha cubierto las siguientes funcionalidades:

a) Fundación y autenticación (Sprint 1)

- **Sistema de autenticación por roles:** la plataforma distingue entre Clientes, Restaurantes y Repartidores desde el registro.
- **Registro tripartito:** un único formulario de registro (AuthController@register) que captura la información específica requerida para cada uno de los tres roles.
- **Login seguro:** un formulario de inicio de sesión (AuthController@login) que valida credenciales y utiliza contraseñas encriptadas (Hash).

b) Migración a Laravel y gestión de restaurantes (Sprint 2)

- **Dashboards dinámicos (AJAX):** se implementaron los paneles de control (cliente-dashboard, restaurante-dashboard, repartidor-dashboard) que cargan las diferentes secciones de forma asíncrona (public/js/dashboard.js).
- **Gestión completa de menú (CRUD):** los restaurantes pueden gestionar su menú (PlatilloController). Tienen la capacidad de añadir, editar y eliminar platillos (con nombre, descripción, precio, categoría e imagen).
- **Filtros dinámicos:** los clientes pueden filtrar los platillos disponibles por categoría o popularidad en la sección "ordenar".

c) Ciclo de vida completo del pedido (Sprint 3)

- **Flujo de cliente (carrito y pago):**
 - Implementación de un sistema de carrito de compras y creación de pedidos (PedidoController@store).
 - Pasarela de pago simulada (PaymentController) que utiliza detalles de tarjeta almacenados en el perfil del cliente.
 - Cálculo de costos (subtotal, costo de envío y comisión de plataforma).

- **Flujo de restaurante (gestión de órdenes):**
 - El dashboard del restaurante (RestauranteDashboardController) muestra los pedidos entrantes.
 - El restaurante puede aceptar, rechazar o actualizar el estado de los pedidos (ej. "en preparación", "listo para recoger").
- **Flujo de repartidor (gestión de entregas):**
 - El dashboard del repartidor (RepartidorController) muestra los pedidos listos para recoger.
 - El repartidor puede aceptar un pedido y actualizar su estado ("en camino", "entregado").
- **Seguimiento de pedidos:**
 - El cliente puede rastrear el estado de su pedido en tiempo real desde su dashboard (ClienteController@getPedidos).
 - El cliente puede cancelar pedidos que aún estén en estado "pendiente".

d) Gestión avanzada de perfiles (Post-Sprint 3)

- Se crearon controladores dedicados para la gestión de perfiles de cada rol (ClienteProfileController, RestauranteProfileController, RepartidorProfileController).
- Los restaurantes pueden gestionar detalles adicionales de su negocio (RestaurantDetail), como descripciones, horarios y banners.
- Los clientes pueden gestionar de forma segura sus datos de pago (dirección, nombre en la tarjeta, últimos 4 dígitos y fecha de expiración).

e) Módulo de preguntas frecuentes (FAQ) (Post-Sprint 3)

- Se implementó un módulo de soporte (FaqController, modelo Faq).

- Este módulo permite a los usuarios consultar una lista de preguntas y respuestas comunes, cargadas desde la base de datos.

4. Estructura de la base de datos (migraciones)

El estado actual de la base de datos está definido por las siguientes migraciones principales:

- 0001_01_01_000000_create_users_table: crea la tabla users con los campos base (nombre, email, password, role).
- 2025_10_18_010116_create_faq_table: crea la tabla faq (pregunta, respuesta).
- 2025_10_19_234145_create_platillos_table: crea la tabla platillos (user_id, nombre, descripcion, precio, imagen, categoria).
- 2025_10_21_191953_create_restaurant_details_table: tabla para información extendida de restaurantes.
- 2025_10_27_000316_create_pedidos_table: tabla pedidos (cliente_id, restaurante_id, repartidor_id, estado, total).
- 2025_10_27_023440_create_detalle_pedidos_table: tabla detalle_pedidos (pedido_id, platillo_id, cantidad, precio).

Además, se utilizaron migraciones para añadir columnas y refinar las tablas existentes:

- Adición de dirección (address) a users.
- Adición de detalles de pago (card_last_four, card_expiry, card_name) a users.
- Adición de columnas de precios (subtotal, costo_envio, comision_plataforma) a pedidos.

5. Conclusión

Mercy Food ha alcanzado un estado de prototipo funcional y robusto. La plataforma soporta el flujo transaccional completo, desde el registro de tres tipos de usuarios hasta la entrega final de un pedido, pasando por la gestión de menús, perfiles y pagos.

La arquitectura en Laravel 12 y el uso de carga asíncrona (AJAX) proporcionan una base sólida, escalable y una experiencia de usuario moderna.